# Automata Theory

## Non-Deterministic to Deterministic

CS32022

**By Thisara Watawana**

2 September 2026

# 4. Non-Deterministic to Deterministic

- NFAs offer parallel execution, implicit rejection, and ε-transitions, making complex machine assembly far easier.
- Does this extra flexibility allow NFAs to recognise brand-new languages that no DFA could ever recognise?
- **Every Non-Deterministic Finite Automaton has an equivalent Deterministic Finite Automaton.**
- Nondeterminism changes design convenience and state efficiency, but not absolute computational power: **L(NFA) = L(DFA)**.

# 4. Non-Deterministic to Deterministic

## The Subset Construction

- A DFA must track all possible parallel execution paths of an NFA using a single deterministic state.
- Each state in the new DFA represents a set of states that the NFA could simultaneously occupy after reading a given input string.
- If an NFA has n states (|Q| = n), the equivalent DFA's states are drawn from the Power Set $\mathscr{P}(Q)$.
    - Maximum possible DFA states = $2^n$ (though many subsets are often unreachable in practice).
    - A DFA state is an accept state if it contains at least one accept state from the original NFA.

# 4. Non-Deterministic to Deterministic

## The ε-Closure Operator: E(R)

- For any set of NFA states $R \subseteq Q$, the ε-closure E(R) is the set of all states reachable from R by taking 0 or more ε-transitions.
- Key Rules of E(R):
  - Self-Inclusion: Every state in R is automatically in E(R) (0 steps taken).
  - Propagation: If state $q \in E(R)$ and $q \xrightarrow{\varepsilon} q'$, then $q' \in E(R)$.
- When to Apply E(R):
  - At Start: To determine the DFA start state $q_0'$.
  - After Transitions: Every time a symbol is read, instantly calculate E(R) on the resulting state set to absorb all free jumps.

# 4. Non-Deterministic to Deterministic

**When to Apply E(R):**

- Machine Initialization (Finding DFA Start State $q_0'$)
  - Before reading any input symbols, calculate $E(\{q_0\})$.
  - Captures all states reachable from original start state $q_0$ via zero or more ε-transitions.
  - Result: DFA Start State $q_0' = E(\{q_0\})$
- Evaluating Input Transitions (Finding Next DFA State)
  - Step A (Consume Symbol): Collect all immediate target states reachable on symbol 'a' from current set R:$M = \cup_{q \in R} \delta(q, a)$
  - Step B (Apply Closure): Instantly expand set M to include all secondary free jumps:$\delta'(R, a) = E(M)$

# 4. Non-Deterministic to Deterministic

## Subset Construction

Given an NFA $M = (Q, \Sigma, \delta, q_0, F)$, we construct an equivalent DFA $M' = (Q', \Sigma, \delta', q_0', F')$ as follows:

1. DFA States: $Q' = \mathscr{P}(Q)$
2. DFA Start State: $q_0' = E(\{q_0\})$
3. DFA Transition Function $\delta'$:
    - For any DFA state $R \in Q'$ and symbol $a \in \Sigma$:
    - $\delta'(R, a) = E( \cup_{q \in R} \delta(q, a) )$
4. DFA Accept States:
    - $F' = \{ R \in Q' \mid R \cap F \neq \varnothing \}$

# 4. Non-Deterministic to Deterministic

## Exercise 06:

Given NFA M over $\Sigma = \{0, 1\}$:

- $Q = \{q_0, q_1, q_2\}$
- $q_0 = q_0$, $F = \{q_2\}$
- Transitions:
  - $\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_1\}$, $\delta(q_0, \varepsilon) = \varnothing$
  - $\delta(q_1, 0) = \varnothing$, $\delta(q_1, 1) = \{q_2\}$, $\delta(q_1, \varepsilon) = \{q_2\}$
  - $\delta(q_2, 0) = \{q_0\}$, $\delta(q_2, 1) = \varnothing$, $\delta(q_2, \varepsilon) = \varnothing$

Tasks:

- Compute $E(\{q_0\})$.
- Construct the transition table for the equivalent DFA.
- Identify all final accept states F' in the resulting DFA.

# 4. Non-Deterministic to Deterministic

## Exercise 06: Solution

- DFA Start State: $q_0' = E(\{q_0\}) = \{q_0\}$

| DFA State R | Input 0 | Input 1 | Accept State? |
|---|---|---|---|
| **$\{q_0\}$** | $\{q_0, q_1, q_2\}$ | $\{q_1, q_2\}$ | No |
| **$\{q_0, q_1, q_2\}$** | $\{q_0, q_1, q_2\}$ | $\{q_1, q_2\}$ | **YES** (contains $q_2$) |
| **$\{q_1, q_2\}$** | $\{q_0\}$ | $\{q_2\}$ | **YES** (contains $q_2$) |
| **$\{q_2\}$** | $\{q_0\}$ | $\varnothing$ | **YES** (contains $q_2$) |
| $\varnothing$ | $\varnothing$ | $\varnothing$ | No |

# 4. Regular Expressions

## Introduction to Regular Expressions

- Declarative vs. Imperative:
  - Automata (DFAs/NFAs) describe computation by how a machine steps through state transitions.
  - Regular Expressions (Regex) describe computation by what pattern the language strings look like.
- Modern pattern matchers in Python, JavaScript, Linux (grep, awk), and compiler lexers use regular expressions to search, validate, and parse text streams.
- Every Regular Expression can be converted into an equivalent NFA, proving that Regexes describe the exact same family of Regular Languages.

# 4. Regular Expressions

## Formal Definition of Regular Expressions

R is a regular expression over alphabet $\Sigma$ if R is:

- a for some a $\in \Sigma$ (Atomic character matching)
- $\varepsilon$ (Matches the empty string)
- $\varnothing$ (Matches the empty language; matches no string)
- $(R_1 \cup R_2)$ (Union: Matches strings in $R_1$ or $R_2$)
- $(R_1 \circ R_2)$ (Concatenation: Matches a string from $R_1$ followed by $R_2$)
- $(R_1^*)$ (Kleene Star: Matches 0 or more repetitions of $R_1$)

# 4. Regular Expressions

## Operator Precedence & Common Shorthands

- Operator Precedence (Highest to Lowest):
  - Star (*) (Binds tightest)
  - Concatenation (◦)
  - Union (∪) (Lowest priority)
  - Example: a ∪ bc* is evaluated as a ∪ (b ◦ (c*))
- Common Syntactic Sugar (Shorthands):
  - $R^+$ = R R* (One or more repetitions)
  - R? = R ∪ ε (Optional element; zero or one match)
  - Σ = (0 ∪ 1) (Any single character in the alphabet)

# 4. Regular Expressions

**Exercise 07:**

Let Σ = {0, 1}. Describe the language denoted by each regular expression:

- 0*10*
- (0 ∪ 1)* 11 (0 ∪ 1)*
- (00 ∪ 11)*
- (0 ∪ ε)(1 ∪ ε)

Tasks:

- Take 3 minutes to write down English descriptions or example strings for each language.

# 4. Regular Expressions

**Exercise 08:**

Let $\Sigma = \{0, 1\}$. Write regular expressions for the following languages:

- $L_1 = \{ w \mid w \text{ starts with 1 and ends with 0 } \}$
- $L_2 = \{ w \mid w \text{ contains an odd number of 1s } \}$
- $L_3 = \{ w \mid w \text{ does not contain the substring 11 } \}$

Tasks:

- Take 4 minutes to draft these expressions.

# 4. Regular Expressions

**Exercise 08:**

Let $\Sigma = \{0, 1\}$. Write regular expressions for the following languages:

- $L_1 = \{ w \mid w \text{ starts with 1 and ends with 0 } \}$
- $L_2 = \{ w \mid w \text{ contains an odd number of 1s } \}$
- $L_3 = \{ w \mid w \text{ does not contain the substring 11 } \}$

Tasks:

- Take 4 minutes to draft these expressions.

# 4. Regular Expressions

## Equivalence of Regex and Automata

- Kleene's Theorem:
  - A language is recognized by a finite automaton (DFA/NFA) if and only if it can be described by a Regular Expression.
- The Two Directions:
  - Regex ⟶ NFA: Convert any regex into an equivalent NFA (Structural Induction).
  - DFA ⟶ Regex: Convert any DFA into an equivalent regex (GNFA / State Elimination).

# 4. Regular Expressions

## Equivalence of Regex and Automata

To convert any regex R into an NFA N:

- Base Cases (Atomic Expressions):
  - R = a: ⟶ $(q_0)$ ⟶a⟶ $((q_1))$
  - R = ε: ⟶ $((q_0))$
  - R = ∅: ⟶ $(q_0)$ (No transitions, no accept states)
- Inductive Steps (Operators):
  - $R_1 \cup R_2$: Apply NFA Union construction.
  - $R_1 \circ R_2$: Apply NFA Concatenation construction.
  - $R_1^*$: Apply NFA Kleene Star construction.

# 4. Regular Expressions

## Equivalence of Regex and Automata

To convert any regex R into an NFA N:

- Base Cases (Atomic Expressions):
  - R = a: ⟶ $(q_0)$ ⟶a⟶ $((q_1))$
  - R = ε: ⟶ $((q_0))$
  - R = ∅: ⟶ $(q_0)$ (No transitions, no accept states)
- Inductive Steps (Operators):
  - $R_1 \cup R_2$: Apply NFA Union construction.
  - $R_1 \circ R_2$: Apply NFA Concatenation construction.
  - $R_1^*$: Apply NFA Kleene Star construction.

# 4. Regular Expressions

## Exercise 09:

Construct NFAs for the following regular expressions using the formal inductive rules (base cases + operators):

- **$R_1 = (0 \cup 1)0$**
  - Focus: Combining Union followed by Concatenation.
- **$R_2 = (01)^*$**
  - Focus: Combining Concatenation wrapped in Kleene Star.

Take 4 minutes to draw the atomic sub-machines first, then glue them together using ε-transitions.

# 4. Regular Expressions

## Exercise 09:

Construct NFAs for the following regular expressions using the formal inductive rules (base cases + operators):

- **$R_1 = (0 \cup 1)0$**
  - Focus: Combining Union followed by Concatenation.
- **$R_2 = (01)^*$**
  - Focus: Combining Concatenation wrapped in Kleene Star.

Take 4 minutes to draw the atomic sub-machines first, then glue them together using ε-transitions.

# 4. Regular Expressions

## Exercise 09 Solution:

Solution 1: (0 ∪ 1)0

- Build atomic machines for 0 and 1.
- Create new start state $q_0$ with ε-branches to both 0 and 1 (Union).
- Connect old accept states of (0 ∪ 1) via ε-arrows to the start state of the final 0 machine (Concatenation).

Solution 2: (01)*

- Build atomic machine for 01 by chaining 0 —ε—► 1 via concatenation.
- Add new start state S_{new} (Accept state) with an ε-arrow to the start of 01 (Handles ε).
- Add a feedback ε-arrow from the end of 01 back to its start state (Handles loops).

# Questions?

thisaraw@boffin.lk